

# Graph Search and Evolutionary Optimization for Automated Model Selection Across Regression and Image Classification

Naga Sampath Maddineni  
University of North Texas  
Denton, Texas, USA

## Abstract

Model selection is often implemented as disconnected trial-and-error experiments. This project develops a common optimization layer for two learning tasks that originally appeared as separate coursework notebooks: a small tabular housing-price regression task and a PyTorch multilayer perceptron for MNIST digit classification. A discrete hyperparameter space is represented as a graph whose nodes are complete configurations and whose edges change one parameter. Best-first graph search provides an informed traversal baseline, while an elitist genetic algorithm provides population-based exploration. A locally executed housing experiment selected a learning rate of 0.005, 500 epochs, and L2 regularization of 0.01, producing an RMSE of \$8,959 on a fixed two-example holdout from the eight-example pedagogical dataset. The saved MNIST notebook provides a baseline of 97.80% test accuracy after five epochs. The integrated MNIST path is executable after installing the declared PyTorch dependencies.

**Index Terms:** *automated model selection, graph search, genetic algorithms, hyperparameter optimization, regression, image classification, MNIST*

## I. Introduction

Machine-learning projects frequently separate data preparation, model training, and hyperparameter selection into disconnected scripts. This separation makes it difficult to compare search strategies or reproduce why a configuration was selected. The repository used for this study contained two independent Fundamentals of Artificial Intelligence exercises: a gradient-descent housing-price regression notebook and a PyTorch MLP notebook for MNIST.

This work turns those exercises into one project. A node represents a complete hyperparameter configuration, and an edge connects configurations that differ in exactly one parameter. The shared interface allows graph search and evolutionary optimization to operate over both tasks while keeping data loading and model training task-specific.

The research question is whether one inspectable search-and-optimization controller can select useful configurations across tabular regression and image classification. The contributions are a reusable discrete search-space abstraction, best-first search, an elitist genetic optimizer with caching and convergence traces, task adapters for the two notebooks, and a single JSON result schema.

## II. Problem Statement and Related Work

Let  $H$  denote a finite set of hyperparameter configurations and let  $J(h)$  be a task-specific validation objective. The selection problem is:

$$h^* = \arg \min J(h), h \in H$$

For regression,  $J$  is holdout mean squared error. For classification,  $J$  is validation cross-entropy loss. The search trace must remain auditable: a reviewer should see evaluated configurations, evaluation counts, and the incumbent score over time.

Graph search explores structured state spaces through frontier-ordering rules such as breadth-first, depth-first, and best-first search [1]. Here the state space is a Cartesian product of hyperparameter values. Genetic algorithms maintain a population of candidates and use selection, crossover, and mutation without requiring a differentiable objective [2]. The learning tasks use established gradient-descent and backpropagation methods [3], and MNIST remains a controlled digit-recognition benchmark [4]. This project does not claim a new model architecture. Its contribution is the explicit optimization layer shared by both tasks.

## III. Coursework Reframing

The first notebook one-hot encoded location, standardized size, bedrooms, bathrooms, garage count, and age, then trained a linear model with gradient descent. Its saved output predicted \$425,005.21 for a fixed Downtown query house. Because the source table has only eight rows and no holdout protocol, this prediction is retained as baseline evidence rather than a generalization result.

The second notebook loaded MNIST with torchvision, split the training set into 50,000 training and 10,000 validation examples, and trained a three-hidden-layer MLP with dropout. Its saved output reported 97.80% test accuracy after five epochs. The integrated adapter exposes hidden widths, dropout, learning rate, and epoch count to the search layer.

The notebooks are therefore not presented as one pre-existing model. They become task implementations under a common question: can graph search and evolutionary optimization select configurations across heterogeneous supervised-learning tasks?

## IV. Methodology

### A. Configuration Graph

A search space maps parameter names to finite option lists. A neighbor changes one parameter to another allowed value while keeping all remaining values fixed. The graph is generated lazily, so no explicit adjacency matrix is required.

### B. Best-First Search

Best-first search evaluates a starting configuration and places its neighbors in a priority queue keyed by objective value. The lowest-scoring frontier node is expanded next. A cache ensures that a configuration is evaluated at most once. The result contains the best configuration found within a budget and the incumbent score after each expansion.

### C. Genetic Optimization

The genetic optimizer samples an initial population. Each generation evaluates and ranks the population, retains an elite fraction, creates children by uniform crossover, mutates parameters, and records the best score. A deterministic seed of 42 and evaluation caching make the experiment reproducible.

### D. Housing Regression Task

Location is one-hot encoded into three categories, numerical variables are standardized using the training split, and prices are scaled by 100,000 during optimization. The model uses batch gradient descent with optional L2 regularization. Six rows form the training split and two rows form the fixed holdout split.

TABLE I. HOUSING SEARCH SPACE

| Parameter      | Values            |
|----------------|-------------------|
| Learning rate  | 0.005, 0.01, 0.02 |
| Epochs         | 500, 1000, 1500   |
| L2 coefficient | 0, 0.001, 0.01    |

### E. MNIST Classification Task

The MNIST adapter uses a fully connected MLP with flattening, two configurable hidden layers, ReLU activations, dropout, and a ten-class output layer. Adam minimizes cross-entropy loss. The task supports a reduced training subset for affordable search and a final test evaluation. Downloaded data and model checkpoints remain local. The search space contains hidden-layer widths of 128 or 256 and 64 or 128, dropout values of 0.2, 0.3, or 0.4, learning rates of 0.0005 or 0.001, and three or five epochs.

## V. Experimental Protocol

All JSON records include the project name, random seed, optimizer method, selected configuration, scalar objective, evaluation count, and convergence history. The housing path was executed locally with the bundled NumPy runtime, and Python bytecode compilation completed for all new source files.

The MNIST path is executable with the declared torch and torchvision dependencies. The validation environment did not include that optional training stack, so this report uses the saved notebook output as the MNIST baseline and does not invent a new genetic-search accuracy.

## VI. Results

Both housing optimizers selected learning rate 0.005, 500 epochs, and L2 coefficient 0.01. The fixed two-row holdout RMSE was \$8,959.05. Best-first search used 18 unique objective evaluations, while the genetic run used 12 unique configurations.

TABLE II. REPRODUCED HOUSING SEARCH RESULTS

| Method     | Learning rate | Epochs | Holdout RMSE |
|------------|---------------|--------|--------------|
| Best-first | 0.005         | 500    | \$8,959.05   |
| Genetic    | 0.005         | 500    | \$8,959.05   |

The agreement is a consistency check, not evidence that either optimizer is superior. The two-row test set and small search space make this experiment a reproducible integration test.

The original MNIST notebook recorded 95.36% validation accuracy after epoch one and 97.60% after epoch five. Its saved test output was 97.80% accuracy with a test loss of 0.0739. These values establish a baseline for the integrated adapter. A fresh optimizer comparison should repeat each configuration across multiple seeds.

## VII. Discussion

The main result is architectural rather than a state-of-the-art score. The repository now separates search policy from task evaluation, so a new model or dataset can be added without rewriting the optimizer. The limitations remain visible: the housing task is too small for a strong statistical conclusion, and the MNIST optimizer has not been rerun in the current validation environment.

Best-first search exploits objective values as soon as neighboring nodes are evaluated. The genetic method maintains population diversity and can combine parameter values that were not adjacent in the search graph. On the housing smoke test, both reached the same configuration, which is expected in a small finite space and does not establish general dominance.

## VIII. Limitations and Threats to Validity

Eight housing examples cannot support a meaningful claim about real-world pricing. The result should be replaced with a public dataset, documented preprocessing, and train, validation, and test splits before the project is used as an empirical housing study.

The saved MNIST result comes from the original notebook, whereas the newly integrated MNIST optimizer was not executed in the current environment. Direct comparisons must wait for a fresh run under the same hardware, data subset, seeds, and budget. The search-space design is also intentionally small and discrete.

## IX. Conclusion

This work converts two independent AI coursework notebooks into a coherent project on automated model selection. A discrete configuration graph, best-first search, and genetic optimization now share one result schema and operate over both a gradient-descent regression task and a PyTorch MLP task. The locally reproduced housing experiment selected the same configuration with both optimizers and achieved a \$8,959 holdout RMSE on the pedagogical two-row split. The saved MNIST baseline provides a 97.80% test-accuracy reference for future integrated runs.

The next empirical step is to execute the MNIST branch across repeated seeds, add a public housing dataset, and report confidence intervals rather than relying on a single small smoke test.

## X. References

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.
- [2] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, 1989.
- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, Learning representations by back-propagating errors, *Nature*, vol. 323, pp. 533-536, 1986.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, Gradient-based learning applied to document recognition, *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.
- [5] Y. LeCun, C. Cortes, and C. J. C. Burges, The MNIST database of handwritten digits, online resource.